

Serializability in DBMS

K Krishna priya
CSE department

Serializability

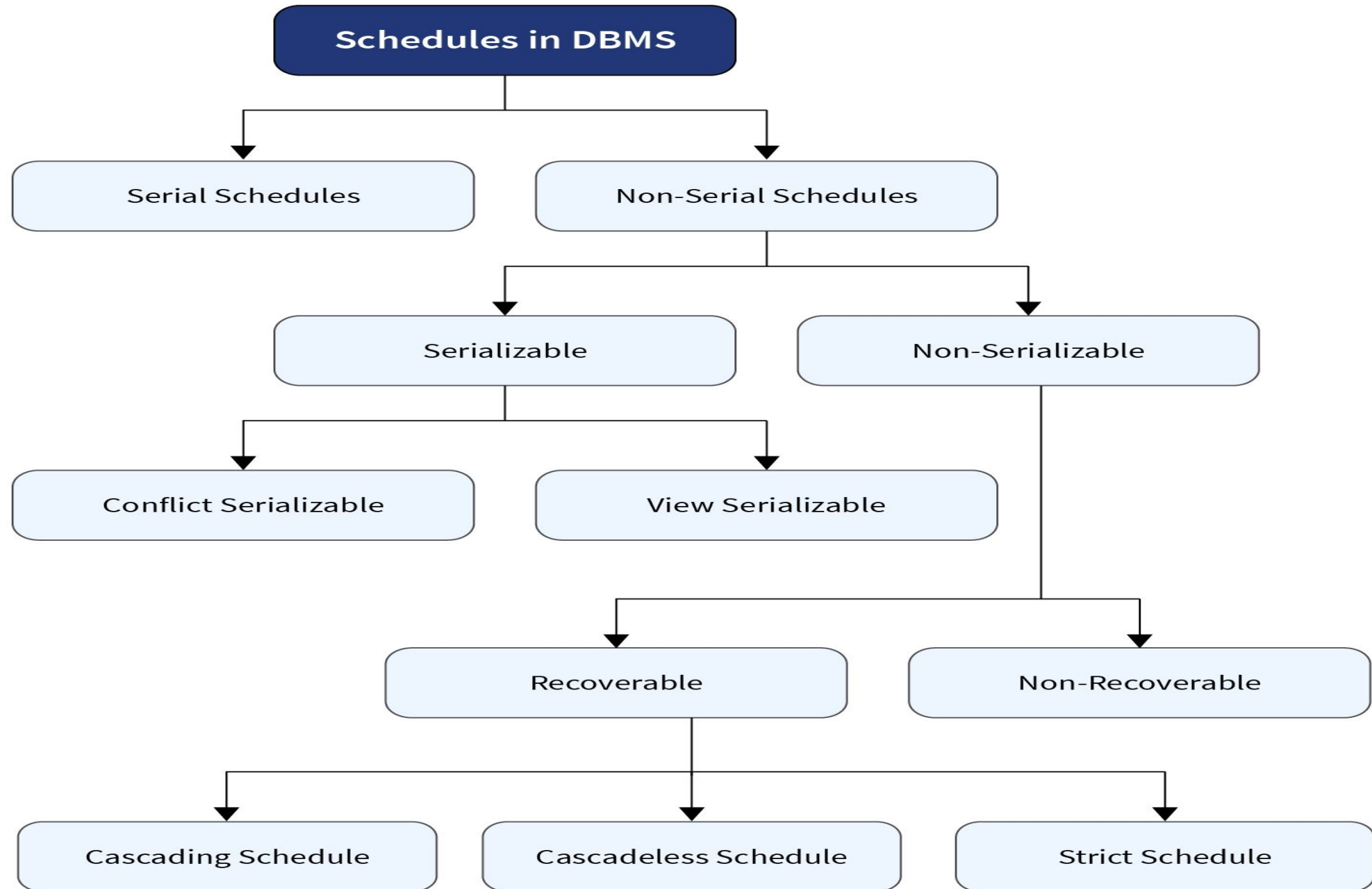
- **Serializability in DBMS ensures that multiple transactions can access and modify the same data without interfering with each other's operations.**
- It helps to prevent data inconsistencies and anomalies that can occur when multiple transactions try to access and modify the same data concurrently

Scheduling

- Scheduling is the technique of preserving the order of the operations from one transaction to another while executing such concurrent transactions.
- A series of operations from one transaction to another transaction is known as a schedule.
- Transactions are a set of instructions that perform operations on databases.
- When multiple transactions are running concurrently, then a sequence is needed in which the operations are to be performed because *at a time, only one operation can be performed on the database*.
- This sequence of operations is known as Schedule, and this process is known as Scheduling.

Types of Schedules

- When multiple transactions execute simultaneously in an unmanageable manner, then it might lead to several problems,
- which are known as concurrency problems. In order to overcome these problems, scheduling is required.
- There are mainly two types of scheduling -
- Serial Schedule
- Non-serial Schedule Further, they are divided into their subcategories, as shown below.



Serial Schedule

- Schedules in which the transactions are executed non-interleaved, i.e.
- As the name says, all the transactions are executed serially one after the other.
- In serial Schedule, a transaction does not start execution until the currently running transaction finishes execution.
- This type of execution of the transaction is also known as non-interleaved execution.
- Serial Schedules are always recoverable, cascades, strict, and consistent. A serial schedule always gives the correct result.
- **Example:** Consider the following schedule involving two transactions T_1 and T_2 .
- where $R(A)$ denotes that a read operation is performed on some data item 'A' This is a serial schedule since the transactions perform serially in the order $T_1 \rightarrow T_2$

T_1	T_2
R(A)	
W(A)	
R(B)	
	W(B)
	R(A)
	R(B)

Non-Serial Schedule:

- This is a type of Scheduling where the operations of multiple transactions are interleaved.
- This might lead to a rise in the concurrency problem. The transactions are executed in a non-serial manner, keeping the end result correct and same as the serial schedule.
- serial schedule where one transaction must wait for another to complete all its operation,
- in the non-serial schedule, the other transaction proceeds without waiting for the previous transaction to complete.
- We can observe that Transaction-2 begins its execution before Transaction-1 is finished, and they are both working on the same data, i.e., “a” and “b”, interchangeably. Where “R”-Read, “W”-Write

Transacti on-1	Transacti on-2
R(a)	
W(a)	
	R(b)
	W(b)
R(b)	
	R(a)
W(b)	
	W(a)

Cont..

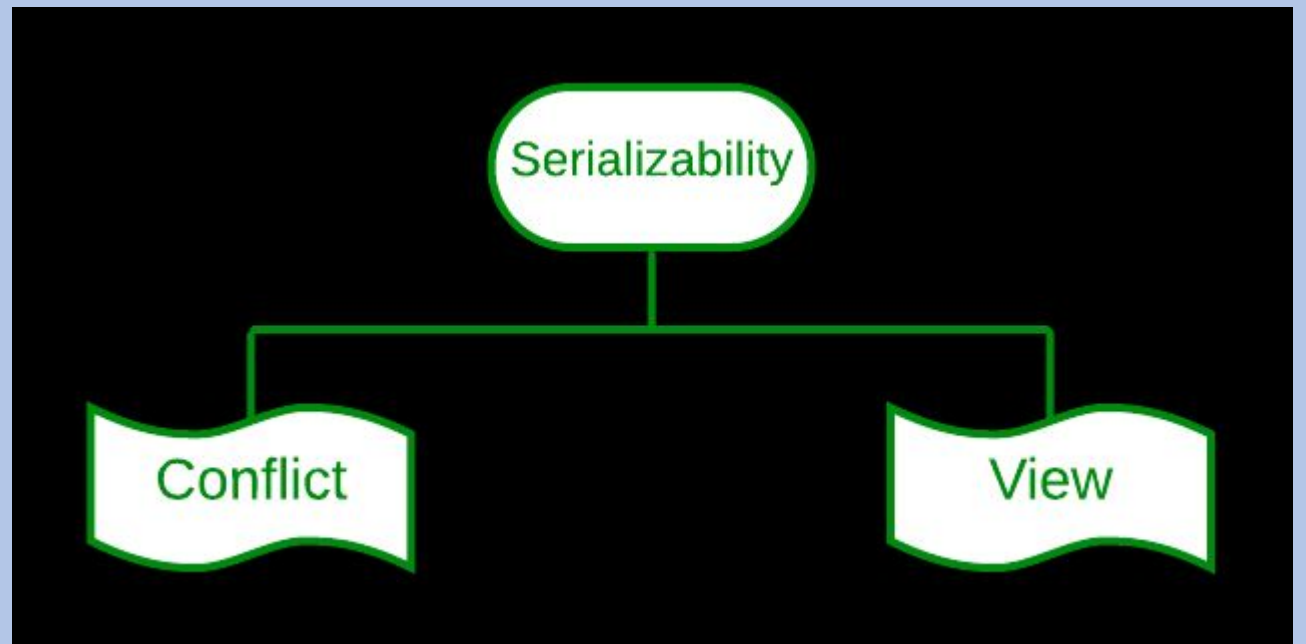
- A schedule in which the transactions are interleaving or interchanging. There are several transactions executing simultaneously as they are being used in performing real-world database operations.
- These transactions may be working on the same piece of data. Hence, the serializability of non-serial schedules is a major concern so that our database is consistent before and after the execution of the transactions.
- It can be of two types namely,
 - **1. Serializable**
 - 2. Non-Serializable Schedule.**

What is Serializability in DBMS?

- Serializability in DBMS ensures safe parallel execution of transactions by enforcing rules, preventing unexpected or incorrect results.
- A non-serial schedule is called a serializable schedule if it can be converted to its equivalent serial schedule. In simple words, if a non-serial schedule and a serial schedule result in the same then the non-serial schedule is called a serializable schedule.
- DBMS employs a technique called concurrency control to achieve serializability.
- This technique helps ensure that multiple transactions do not interfere with each other's execution by controlling their access to shared resources. These shared resources can be database tables and records

Types of Serializability in DBMS

- In DBMS (Database Management System), there are two types of serializability:
- **Conflict Serializability**
- **View Serializability**



Conflict Serializability in DBMS

- As discussed in Concurrency control, serial schedules have less resource utilization and low throughput.
- To improve it, two or more transactions are run concurrently. However, concurrency of transactions may lead to inconsistency in the database. To avoid this, we need to check whether these concurrent schedules are serializable or not.
- **Conflict serializable schedules:** A schedule is called conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.
- **Non-conflicting operations:** When two operations operate on separate data items or the same data item but at least one of them is a read operation, they are said to be non-conflicting.

Conflict Serializability

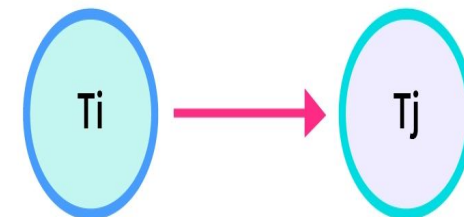
- Conflict serializability is a property of a schedule. It ensures that it is equivalent to a serial schedule.
- A schedule can be declared as conflict serializable if and only if all conflicting pairs of transactions in the schedule have the same relative order in all serial schedules that are equivalent to the given schedule.
- A **conflicting pair of transactions** is a pair of transactions that access the same data item, and at least one of them is a write operation.
- This type of serializability can be determined by drawing a precedence graph of the transactions in the schedule and checking if the graph is acyclic.
- If there is a cycle present in the serialized graph then the schedule is non-serializable because the cycle resembles that one transaction is dependent on the other transaction and vice versa.
- It also means that there are one or more conflicting pairs of operations in the transactions. On the other hand, no-cycle means that the non-serial schedule is serializable.

Testing of Serializability

- To test the serializability of a schedule, we can use **Serialization Graph** or **Precedence Graph**. A serialization Graph is nothing but a Directed Graph of the entire transactions of a schedule.
- It can be defined as a Graph $G(V, E)$ consisting of a set of directed-edges $E = \{E_1, E_2, E_3, \dots, E_n\}$ and a set of vertices $V = \{V_1, V_2, V_3, \dots, V_n\}$. The set of edges contains one of the two operations - READ, WRITE performed by a certain transaction.

$T_i \rightarrow T_j$, means Transaction- T_i is either performing read or write before the transaction- T_j .

Precedence Graph for Schedule S



What is a conflicting pair in transactions?

- Two operations inside a schedule are called conflicting if they meet these three conditions:
- They belong to two different transactions.
- They are working on the same data piece.
- One of them is performing the WRITE operation.
- To conclude, let's take two operations on data: "a". The conflicting pairs are:
 - READ(a) - WRITE(a)
 - WRITE(a) - WRITE(a)
 - WRITE(a) - READ(a)

Conflicting Operations cont..

- **Example:**
- **Conflicting** operations pair $(R_1(A), W_2(A))$ because they belong to two different transactions on the same data item A and one of them is a write operation.
- Similarly, $(W_1(A), W_2(A))$ and $(W_1(A), R_2(A))$ pairs are also **conflicting**.
- On the other hand, the $(R_1(A), W_2(B))$ pair is **non-conflicting** because they operate on different data items.
- Similarly, $((W_1(A), W_2(B)))$ pair is **non-conflicting**.

Non-serializable schedule- Example

- Let's take an example of schedule "S" having three transactions t1, t2, and t3 working simultaneously, to get a better understanding.

Non-serializable schedule. R(x) of T1 conflicts with W(x) of T3, so there is a directed edge from T1 to T3. R(y) of T1 conflicts with W(y) of T2, so there is a directed edge from T1 to T2. W(y\ x) of T3 conflicts with W(x) of T1, so there is a directed edge from T3 to T. Similarly, we will make edges for every conflicting pair. Now, as the cycle is formed, the transactions cannot be serializable.

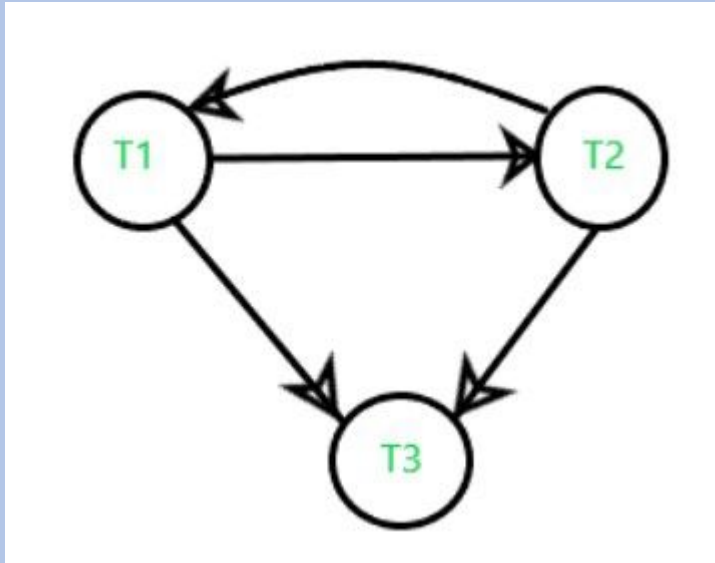
t1	t2	t3
R(x)		
		R(z)
		W(z)
	R(y)	
R(y)		
	W(y)	
		W(x)
	W(z)	
W(x)		

Need of View-Serializability

- There may be some schedules that are not Conflict-Serializable but still gives a consistent result because the concept of [Conflict-Serializability](#) becomes limited when the [Precedence Graph](#) of a schedule contains a **loop/cycle**. In such a case we cannot predict whether a schedule would be consistent or inconsistent.
- But, what if a schedule's precedence graph contains a cycle/loop and is giving consistent result/accurate results as a conflict serializable schedule is giving?
- So, to address such cases we brought the concept of **View-Serializability**

Example: Understanding View-Serializability first with a Schedule S_1 :

So, its Conflict Precedence Graph is as follows –



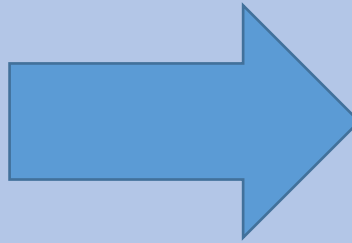
T_1	T_2	T_3
a=100 read(a)		
	a=a-40 write(a) //60	
a=a-40 write(a) //20		
		a=a-20 write(a) //0

The above graph contains cycle/loop which means it is not conflict-serializable but it does not mean that it cannot be consistent and equivalent to the serial schedule it may or may not be.

Look Schedule S'_1 :

In the above example if we do **swapping** among some transaction's operation so our table will look like this

T_1	T_2	T_3
a=100 read(a)		
	a=a-40 write(a) //60	
a=a-40 write(a) //20		
		a=a-20 write(a) //0

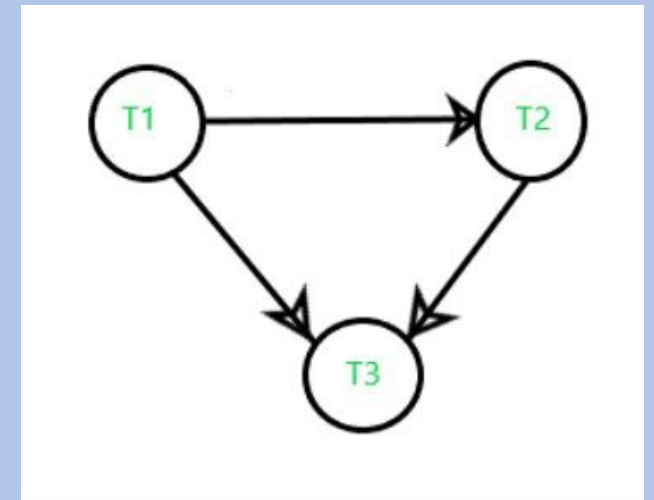


T_1	T_2	T_3
a=100 read(a)		
a=a-40 write(a) //60		
	a=a-40 write(a) //60	
		a=a-20 write(a) //0

Now, we see that the precedence graph of the second table does not contain any cycle/loop, which means it is conflict serializable (equivalent to serial schedule, consistent) and the final result is coming the same as the first table.

Note: In the above example we understood that if a schedule is Conflict-serializable so we can easily predict that It would be –

- Equivalent to a serial schedule,
- Consistent,
- And also a View-Serializable.



View Serializability and View Serializable Schedules

- If a non-serial schedule is **view equivalent** to some other serial schedule then the schedule is called View Serializable Schedule. It is needed to ensure the consistency of a schedule.
- What is view equivalency?
- The two conditions needed by schedules(S1 and S2) to be view equivalent are:
 - Initial read must be on the same piece of data.
 - Example: If transaction t1 is reading "A" from database in schedule S1, then in schedule S2, t1 must read A.
 - Final write must be on the same piece of data.
 - Example: If a transaction t1 updated A at last in S1, then in S2, t1 should perform final write as well.
 - The mid sequence should also be in the same order.
 - Example: If t1 is reading A which is updated by t2 in S1, then in S2, t1 should read A which should be updated by t2.
- This process of checking view equivalency of a schedule is called View Serializability.
- Example: We have a schedule "S" having two transactions t1, and t2 working simultaneously.
- his process of checking view equivalency of a schedule is called View Serializability.

Example: We have a schedule "S" having two transactions t1, and t2 working simultaneously.

t1	t2
R(x)	
W(x)	
R(y)	
W(y)	
	R(x)
	W(x)
	R(y)
	W(y)

t1	t2
R(x)	
W(x)	
	R(x)
	W(x)
R(y)	
W(y)	
	R(y)
	W(y)

Since a view equivalent schedule is possible, it is a view serializable schedule.

View Serializability

- A schedule will view serializable if it is view equivalent to a serial schedule.
- If a schedule is conflict serializable, then it will be view serializable.
- The view serializable which does not conflict serializable contains blind writes.
- **View Equivalent**
 - Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions:
 - **1. Initial Read 2. Updated Read 3. Final Write**

1. Initial Read

- An initial read of both schedules must be the same. Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

T1	T2
Read(A)	Write(A)

Schedule S1

T1	T2
Read(A)	Write(A)

Schedule S2

Above two schedules are view equivalent because Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

2. Updated Read

- In schedule S1, if T_i is reading A which is updated by T_j then in S2 also, T_i should read A which is updated by T_j .

T1	T2	T3
Write(A)	Write(A)	Read(A)

Schedule S1

T1	T2	T3
Write(A)	Write(A)	<u>Read(A)</u>

Schedule S2

Above two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

3. Final Write

- A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S1

T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S2

Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

Example:

With 3 transactions, the total number of possible schedule

$$= 3! = 6$$

$S1 = \langle T1 \ T2 \ T3 \rangle$

$S2 = \langle T1 \ T3 \ T2 \rangle$

$S3 = \langle T2 \ T3 \ T1 \rangle$

$S4 = \langle T2 \ T1 \ T3 \rangle$

$S5 = \langle T3 \ T1 \ T2 \rangle$

$S6 = \langle T3 \ T2 \ T1 \rangle$

Schedule S

T1	T2	T3
Read(A)	Write(A)	Write(A)
Write(A)		

Taking first schedule S1:

Schedule S1

Step 1: final updation on data items

In both schedules S and S1, there is

no read except the initial read that's why we don't need to check that condition.

Step 2: Initial Read

The initial read operation in S is done by T1 and in S1, it is also done by T1.

Step 3: Final Write The final write operation in S is done by T3 and in S1, it is also done by T3. So, S and S1 are view Equivalent.

The first schedule S1 satisfies all three conditions, so we don't need to check another schedule.

Hence, view equivalent serial schedule is: $T1 \rightarrow T2 \rightarrow T3$

T1	T2	T3
Read(A) Write(A)	Write(A)	Write(A)

Difference between Serial Schedule and Serializable Schedule

Serial Schedule	Serializable Schedule
In Serial schedule, transactions will be executed one after other.	In Serializable schedule transaction are executed concurrently.
Serial schedule are less efficient.	Serializable schedule are more efficient.
In serial schedule only one transaction executed at a time.	In Serializable schedule multiple transactions can be executed at a time.
Serial schedule takes more time for execution.	In Serializable schedule execution is fast.